

AI-POWERED CUSTOMER SUPPORT ASSISTANT — INTERVIEW TASK

OVERVIEW

Build a fully offline, local AI-powered customer support assistant using Python (FastAPI), Ollama (local LLM), and Flutter. The system must accept user messages, detect intent via a locally running LLM, execute mock tools, maintain short conversation memory, and return structured JSON responses. The Flutter frontend will dynamically render UI widgets based on the backend's ui_type field.

OBJECTIVE

- Deliver a working prototype that demonstrates:
- Local AI intent classification & tool routing
 - Clean backend architecture with FastAPI
 - Strictly structured JSON API responses
 - Dynamic Flutter UI rendering
 - Short-term conversational context management
 - Fully offline operation (no internet or external AI APIs)

TECHNICAL STACK & CONSTRAINTS

Component	Requirement
Backend	Python, FastAPI
AI/LLM	Ollama (must run locally)
Frontend	Flutter
Network	✗ Strictly offline. No internet access allowed during execution.
External APIs	✗ OpenAI, Claude, or any cloud AI services are prohibited
Allowed Libraries	Standard Python, `requests`/`httpx`, Ollama local REST API

CORE WORKFLOW

1. User types a message in the Flutter chat UI.
2. Frontend sends POST /chat with message & conversation history.
3. Backend forwards context to local Ollama for intent classification & tool selection.
4. Backend executes the selected mock tool, updates memory, and returns structured JSON.
5. Frontend parses ui_type and dynamically renders the corresponding widget.

BACKEND SPECIFICATIONS

Endpoint: POST /chat

Supported Intents & Mock Tools:

- order_tracking → tracking_tool()
- refund_request → refund_tool()
- complaint → complaint_tool()
- escalation → escalation_tool()
- hotel_search → hotel_tool()
- flight_search → flight_tool()

Note: Each tool must return static, predefined data. Do not make external network calls inside tools.

Conversation Memory:

- Maintain short-term context (e.g., last 3–5 turns).
- Must handle follow-up questions referencing prior context.
- Example:

User: "Show hotels in Dubai"

Assistant: Returns hotel list

User: "Show cheaper ones" → System must understand "cheaper ones" refers to the previous hotel search context.

- Simple in-memory storage (list/dict) is sufficient.

Required Response Schema:



```
1 {
2   "intent": "hotel_search",
3   "tool_called": "hotel_tool",
4   "ui_type": "hotel_page",
5   "message": "Available hotels found.",
6   "data": {
7     "hotels": [
8       {
9         "name": "Grand Palace",
10        "price": "$220",
11        "rating": 4.8
12      }
13    ]
14  }
15 }
16
```

Note: Backend must guarantee valid JSON output. Use schema validation (e.g., Pydantic) if possible.

FRONTEND SPECIFICATIONS (FLUTTER)

UI Components:

- Text input field
- Send button
- Chat/message history area
- Dynamic widget rendering container

Rendering Logic:

The app must read `response.ui_type` and render the appropriate widget:

```
1
2  switch (response.uiType) {
3      case "hotel_page":    return HotelWidget(data: response.data);
4      case "flight_page":   return Text("flight_page");
5      case "tracking_page": return Text("tracking_page");
6      case "refund_page":   return Text("refund_page");
7      case "complaint_page": return Text("complaint_page");
8      case "escalation_page": return Text("escalation_page");
9      default:              return Text(response.message);
10 }
```

HotelWidget Requirement:

- Must render a real, structured UI (not a placeholder).
- Use standard Flutter components: Card, ListView, Image (placeholder), Text for name, price, rating.
- Example data structure:

```
1  {
2      "hotels": [
3          { "name": "Grand Palace", "price": "$220", "rating": 4.8 }
4      ]
5  }
6
```

ARCHITECTURE & CODE QUALITY EXPECTATIONS

Candidates should demonstrate:

- Clean architecture & separation of concerns (routes, services, tools, UI)
- Effective prompt engineering for intent/tool routing
- Structured AI workflow & deterministic tool-calling design
- Reliable JSON schema enforcement
- Basic error handling & graceful fallbacks
- Clear frontend/backend integration
- Readable, maintainable, and well-documented code

EVALUATION CRITERIA

Category	What We Look For
Architecture & Maintainability	Modular structure, clear boundaries, scalable patterns
AI Workflow Design	Prompting strategy, tool routing, structured output enforcement
Tool Calling Implementation	Clean mock tool execution, data mapping, error fallbacks
Memory Handling	Context preservation, follow-up comprehension
API Design	Consistent endpoint, typed schemas, predictable JSON
Flutter UI & Dynamic Rendering	Component switching logic, `HotelWidget` implementation
Code Quality & Error Handling	Robustness, async/await usage, logging, fallback UI
Extensibility	Easy to add new intents, tools, or UI types

BONUS (OPTIONAL STRETCH GOALS)

- Streaming responses (SSE or chunked HTTP)
- State management (Bloc, Riverpod, or Provider)
- Dockerized backend setup (docker-compose)
- Typed schemas (Pydantic models, Dart JSON serialization)
- Structured logging & request tracing
- Async tool execution
- Component registry / factory pattern for UI rendering
- Responsive UI layouts

=====

SUBMISSION GUIDELINES

=====

- Provide a public GitHub/GitLab repository link.
- Include a README.md with:
 - Prerequisites & setup steps (Ollama model, Python/Flutter commands)
 - Architecture overview & design decisions
 - API payload examples & curl/Postman instructions
 - Screenshots or short GIF of the Flutter app in action
- Ensure the project runs fully offline after initial dependency/model download.
- (Optional) Include a simple architecture diagram.

=====

IMPORTANT NOTES

=====

- Focus on architecture, AI workflow, and dynamic rendering over UI polish.
- Mock/static data is acceptable only inside tool functions.
- Prioritize deterministic, reliable responses and strict JSON formatting.
- Keep the solution concise but production-ready in structure.
- Interviewers may ask to walk through the prompt design, tool routing logic, and memory implementation during the review.